

C++高级主题

- C++ 11特性
- 异常处理

统一的初始化方法

```
int arr[3]{1, 2, 3};
```

```
vector<int> iv{1, 2, 3};
```

```
map<int, string> mp{{1, "a"}, {2, "b"}};
```

```
string str{"Hello World"};
```

```
int * p = new int[20]{1,2,3};
```

```
struct A {
```

```
    int i,j;  A(int m,int n):i (m),j(n) {
```

```
};
```

```
A func(int m,int n ) { return {m,n}; }
```

```
int main() { A * pa = new A {3,7}; }
```

成员变量默认初始值

```
class B
{
    public:
        int m = 1234;
        int n;
};

int main()
{
    B b;
    cout << b.m << endl;    //输出 1234
    return 0;
}
```

auto关键字

用于定义变量，编译起可以自动判断变量的类型

```
auto i = 100;           // i 是 int
auto p = new A();       // p 是 A *
auto k = 34343LL;       // k 是 long long

map<string,int,greater<string> > mp;
for( auto i = mp.begin(); i != mp.end(); ++i)
    cout << i->first << ", " << i->second ;

//i的类型是:  map<string,int,greater<string> >::iterator
```

auto关键字

```
class A { };

A operator + ( int n,const A & a)
{
    return a;
}

template <class T1, class T2>
auto add(T1 x, T2 y) -> decltype(x + y) {
    return x+y;
}

auto d = add(100,1.5); // d是double d=101.5
auto k = add(100,A()); // k是A类型
```

decltype 关键字

求表达式的类型

```
int i;  
double t;  
struct A { double x; };  
const A* a = new A();
```

```
decltype(a)      x1;    // x1 is A *  
decltype(i)      x2;    // x2 is int  
decltype(a->x)    x3;    // x3 is double  
decltype((a->x)) x4 = t; // x4 is double&
```

智能指针shared_ptr

- 头文件: `<memory>`
 - 通过shared_ptr的构造函数, 可以让shared_ptr对象托管一个new运算符返回的指针, 写法如下:
 - `shared_ptr<T> ptr(new T);` // T 可以是 int ,char, 类名等各种类型
- 此后ptr就可以像 `T*` 类型的指针一样来使用, 即 `*ptr` 就是用new动态分配的那个对象, 而且不必操心释放内存的事。
- 多个shared_ptr对象可以同时托管一个指针, 系统会维护一个托管计数。当无shared_ptr托管该指针时, delete该指针。
 - shared_ptr对象不能托管指向动态分配的数组的指针, 否则程序运行会出错

智能指针shared_ptr

```
#include <memory>
#include <iostream>
using namespace std;

struct A {
    int n;
    A(int v = 0):n(v){ }
    ~A() { cout << n << " destructor" << endl; }
};

int main()
{
    shared_ptr<A> sp1(new A(2)); //sp1托管A(2)
    shared_ptr<A> sp2(sp1);      //sp2也托管 A(2)
    cout << "1)" << sp1->n << ", " << sp2->n << endl;
    //输出1) 2, 2
    shared_ptr<A> sp3;
    A * p = sp1.get();          //p 指向 A(2)
    cout << "2)" << p->n << endl;
```

输出结果:

1) 2, 2

2) 2

```
sp3 = sp1;    //sp3也托管 A(2)
cout << "3)" << (*sp3).n << endl;    //输出 2
sp1.reset();    //sp1放弃托管 A(2)
if( !sp1 )
    cout << "4)sp1 is null" << endl;    //会输出
A * q = new A(3);
sp1.reset(q);    // sp1托管q
cout << "5)" << sp1->n << endl;    //输出 3
shared_ptr<A> sp4(sp1);    //sp4托管A(3)
shared_ptr<A> sp5;
sp1.reset();    //sp1放弃托管 A(3)
cout << "before end main" << endl;
sp4.reset();    //sp4放弃托管 A(3)
cout << "end main" << endl;
return 0;    //程序结束, 会delete 掉A(2)
```

```
}
```

智能指针shared_ptr

```
#include <iostream>
#include <memory>
using namespace std;
struct A{
    ~A() { cout << "~A" << endl; }
};
int main()
{
    A * p = new A();
    shared_ptr<A> ptr(p);
    shared_ptr<A> ptr2;
    ptr2.reset(p); //并不增加 ptr中对p的托管计数
    cout << "end" << endl;
    return 0;
}
```

智能指针shared_ptr

```
#include <iostream>
#include <memory>
using namespace std;
struct A{
    ~A() { cout << "~A" << endl; }
};
int main()
{
    A * p = new A();
    shared_ptr<A> ptr(p);
    shared_ptr<A> ptr2;
    ptr2.reset(p); //并不增加 ptr中对p的托管计数
    cout << "end" << endl;
    return 0;
}
```

输出:

end

~A

~A

之后程序崩溃

因 p被delete两次

空指针nullptr

```
#include <memory>
#include <iostream>
using namespace std;

int main()    {
    int* p1 = NULL;
    int* p2 = nullptr;
    shared_ptr<double> p3 = nullptr;
    if(p1 == p2)
        cout << "equal 1" <<endl;
    if( p3 == nullptr)
        cout << "equal 2" <<endl;
    if( p3 == p2) ; // error
    if( p3 == NULL)
        cout << "equal 4" <<endl;
    bool b = nullptr; // b = false
    int i = nullptr; //error,nullptr不能自动转换成整型
    return 0;
}
```

去掉出错的语句后输出:

```
equal 1
equal 2
equal 4
```

基于范围的for循环

```
#include <iostream>
#include <vector>
using namespace std;
struct A {    int n; A(int i):n(i) {    } };
int main() {
    int ary[] = {1,2,3,4,5};
    for(int & e: ary)
        e*= 10;
    for(int e : ary)
        cout << e << ", ";
    cout << endl;
    vector<A> st(ary,ary+5);
    for( auto & it: st)
        it.n *= 10;
    for( A it: st)
        cout << it.n << ", ";
    return 0;
}
```

输出:

10, 20, 30, 40, 50,
100, 200, 300, 400
, 500,

右值引用和move语义

右值：一般来说，不能取地址的表达式，就是右值，
能取地址的，就是左值

```
class A { };
```

```
A & r = A(); // error , A()是无名变量, 是右值
```

```
A && r = A(); //ok, r 是右值引用
```

主要目的是提高程序运行的效率，减少需要进行深拷贝的对象进行深拷贝的次数。

[参考](#)

<http://amazingjxq.com/2012/06/06/%E8%AF%91%E8%AF%A6%E8%A7%A3c%E5%8F%B3%E5%80%BC%E5%BC%95%E7%94%A8/>

<http://www.cnblogs.com/soaliap/archive/2012/11/19/2777131.html>

```
#include <iostream>
#include <string>
#include <cstring>
using namespace std;

class String
{
public:
    char * str;
    String():str(new char[1]) { str[0] = 0;}
    String(const char * s) {
        str = new char[strlen(s)+1];
        strcpy(str,s);
    }
    String(const String & s) {
        cout << "copy constructor called" << endl;
        str = new char[strlen(s.str)+1];
        strcpy(str,s.str);
    }
}
```

```
String & operator=(const String & s) {  
    cout << "copy operator= called" << endl;  
    if( str != s.str) {  
        delete [] str;  
        str = new char[strlen(s.str)+1];  
        strcpy(str,s.str);  
    }  
    return * this;  
}  
  
    // move constructor  
String(String && s):str(s.str) {  
    cout << "move constructor called"<<endl;  
    s.str = new char[1];  
    s.str[0] = 0;  
}
```

```
// move assignment
```

```
String & operator = (String &&s) {  
    cout << "move operator= called"<<endl;  
    if (str!= s.str) {  
        delete [] str;  
        str = s.str;  
        s.str = new char[1];  
        s.str[0] = 0;  
    }  
    return *this;  
}  
~String() { delete [] str; }  
};
```

```
template <class T>
void MoveSwap(T& a, T& b)    {
    T tmp(move (a)); // std::move(a) 为右值, 这里会调用move
                        constructor
    a = move (b); // move(b) 为右值, 因此这里会调用move assignment
    b = move (tmp); // move(tmp) 为右值, 因此这里会调用move
                        assignment
}
```

```
int main()
{
    //String & r = String("this"); // error
    String s;
    s = String("ok"); // String("ok")是右值
    cout << "*****" << endl;
    String && r = String("this");
    cout << r.str << endl;
    String s1 = "hello", s2 = "world";
    MoveSwap(s1, s2);
    cout << s2.str << endl;
    return 0;
}
```

输出:

move operator= called

.....

this

move constructor called

move operator= called

move operator= called

hello

无序容器(哈希表)

```
#include <iostream>
#include <string>
#include <unordered_map>
using namespace std;
int main()
{
    unordered_map<string,int> turingWinner; //图灵奖获奖名单
    turingWinner.insert(make_pair("Dijkstra",1972));
    turingWinner.insert(make_pair("Scott",1976));
    turingWinner.insert(make_pair("Wilkes",1967));
    turingWinner.insert(make_pair("Hamming",1968));
    turingWinner["Ritchie"] = 1983;
    string name;
    cin >> name; //输入姓名
```

```
    unordered_map<string,int>::iterator p =
turingWinner.find(name);
    //据姓名查获获奖时间
    if( p != turingWinner.end())
        cout << p->second;
    else
        cout << "Not Found" << endl;
    return 0;
}
```

哈希表插入和查询的时间复杂度几乎是常数

正则表达式

```
#include <iostream>
#include <regex> //使用正则表达式须包含此文件
using namespace std;
int main()
{
    regex reg("b.?p.*k");
    cout << regex_match("bopggk", reg) << endl; //输出 1, 表示匹配成功
    cout << regex_match("boopgggk", reg) << endl; //输出 0, 匹配失败
    cout << regex_match("b pk", reg) << endl; //输出 1, 表示匹配成功
    regex reg2("\\d{3}([a-zA-Z]+). (\\d{2} |N/A)\\s\\1");
    string correct="123Hello N/A Hello";
    string incorrect="123Hello 12 hello";
    cout << regex_match(correct, reg2) << endl; //输出 1, 匹配成功
    cout << regex_match(incorrect, reg2) << endl; //输出 0, 失败
}
```

Lambda表达式

只使用一次的函数对象，能否不要专门为其编写一个类？

只调用一次的简单函数，能否在调用时才写出其函数体？

Lambda表达式

形式:

[外部变量访问方式说明符](参数表) -> 返回值类型

```
{  
    语句组  
}
```

[] 不使用任何外部变量

[=] 以传值的形式使用所有外部变量

[&] 以引用形式使用所有外部变量

[x, &y] x 以传值形式使用, y 以引用形式使用

[=, &x, &y] x, y 以引用形式使用, 其余变量以传值形式使用

[&, x, y] x, y 以传值的形式使用, 其余变量以引用形式使用

“->返回值类型”也可以没有, 没有则编译器自动判断返回值类型。

Lambda表达式

```
int main()
{
    int x = 100,y=200,z=300;
    cout << [ ] (double a,double b) { return a + b; } (1.2,2.5)
        << endl;
    auto ff = [=,&y,&z] (int n) {
        cout <<x << endl;
        y++; z++;
        return n*n;
    };
    cout << ff(15) << endl;
    cout << y << "," << z << endl;
}
```

Lambda表达式

```
int main()
{
    int x = 100,y=200,z=300;
    cout << [ ] (double a,double b) { return a + b; } (1.2,2.5)
        << endl;
    auto ff = [=,&y,&z] (int n) {
        cout <<x << endl;
        y++; z++;
        return n*n;
    };
    cout << ff(15) << endl;
    cout << y << ", " << z << endl;
}
```

输出:
3.7
100
225
201,301

Lambda表达式

```
int a[4] = { 4,2,11,33};  
sort(a,a+4, [ ](int x,int y)->bool {  
    return x%10 < y%10; } ) ;  
  
for_each(a,a+4, [ ](int x) {cout << x << " " ;} ) ;
```

Lambda表达式

```
int a[4] = { 4,2,11,33};  
sort(a,a+4, [ ](int x,int y)->bool { return x%10  
                                     < y%10; }));  
for_each(a,a+4, [ ](int x) {cout << x << " " ;} ) ;
```

输出:

11 2 33 4

Lambda表达式

```
#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;
int main()
{
    vector<int> a { 1,2,3,4};
    int total = 0;
    for_each(a.begin(),a.end(), [&] (int & x)
        {total += x; x*=2;});
    cout << total << endl; //输出 10
    for_each(a.begin(),a.end(), [ ] (int x)
        { cout << x << " ";});
    return 0;
}
```

程序输出结果:

10

2 4 6 8

Lambda表达式

实现递归求斐波那契数列第n项:

```
function<int(int)> fib = [&fib](int n)
{ return n <= 2 ? 1 : fib(n-1) + fib(n-2);};
```

```
cout << fib(5) << endl;    //输出5
```

`function<int(int)>` 表示返回值为 `int`, 有一个`int`参数的函数

C++高级主题

- C++ 11特性
- 异常处理

程序运行发生异常

- 程序运行中总难免发生错误
 - 数组元素的下标超界、访问NULL指针
 - 除数为0
 - 动态内存分配new需要的存储空间太大
 -

程序运行发生异常

- 引起这些异常情况的原因：
 - 代码质量不高，存在BUG
 - 输入数据不符合要求
 - 程序的算法设计时考虑不周到
 -

程序运行发生异常

- 发生异常怎么办
 - 不只是简单地终止程序运行
 - 能够反馈异常情况的信息：哪一段代码发生的、什么异常
 - 能够对程序运行中已发生的事情做些处理：取消对输入文件的改动、释放已经申请的系统资源

异常处理

- 一个函数运行期间可能产生异常。在函数内部对异常进行处理未必合适。因为函数设计者无法知道函数调用者希望如何处理异常。
- 告知函数调用者发生了异常，让函数调用者处理比较好
- 用函数返回值告知异常不方便

用try、catch进行异常处理

```
#include <iostream>
using namespace std;
int main()
{
    double m ,n;
    cin >> m >> n;
    try {
        cout << "before dividing." << endl;
        if( n == 0)
            throw -1; //抛出int类型异常
        else
            cout << m / n << endl;
        cout << "after dividing." << endl;
    }
```

```
catch(double d) {  
    cout << "catch(double) " << d << endl;  
}  
catch(int e) {  
    cout << "catch(int) " << e << endl;  
}  
cout << "finished" << endl;  
return 0;  
}
```

程序运行结果如下:

9 6✓

before dividing.

1.5

after dividing.

finished

捕获任何异常的catch块

```
#include <iostream>
using namespace std;
int main()
{
    double m ,n;
    cin >> m >> n;
    try {
        cout << "before dividing." << endl;
        if( n == 0)
            throw -1; //抛出整型异常
        else if( m == 0 )
            throw -1.0; //抛出double型异常
        else
            cout << m / n << endl;
        cout << "after dividing." << endl;
    }
```

```

catch(double d) {
    cout << "catch(double) " << d << endl;
}
catch(...) {
    cout << "catch(...) " << endl;
}
cout << "finished" << endl;
return 0;
}

```

程序运行结果:

9 0 ✓

before dividing.

catch(...)

finished

.0_6_ ✓

before dividing.

catch(double) -1

finished

注意: try块中定义的局部对象, 发生异常时会析构!

异常的再抛出

如果一个函数在执行的过程中，抛出的异常在本函数内就被`catch`块捕获并处理了，那么该异常就不会抛给这个函数的调用者（也称“上一层的函数”）；如果异常在本函数中没被处理，就会被抛给上一层的函数。

异常再抛出

```
#include <iostream>
#include <string>
using namespace std;
class CException
{
    public :
        string msg;
        CException(string s):msg(s) { }
};
```

```
double Devide(double x, double y)
{
    if(y == 0)
        throw CException("devided by zero");
    cout << "in Devide" << endl;
    return x / y;
}

int CountTax(int salary)
{
    try {
        if( salary < 0 )
            throw -1;
        cout << "counting tax" << endl;
    }
    catch (int ) {
        cout << "salary < 0" << endl;
    }
}
```

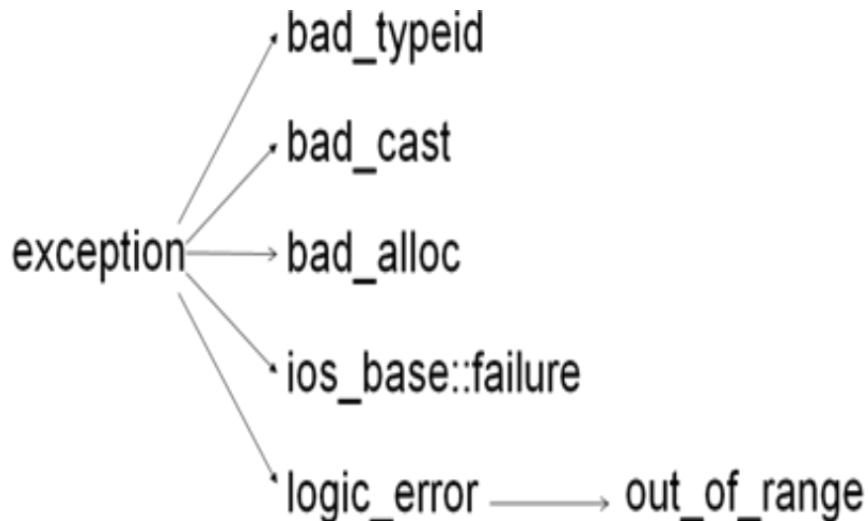
```
        cout << "tax counted" << endl;
        return salary * 0.15;
    }
int main()
{
    double f = 1.2;
    try {
        CountTax(-1);
        f = Devide(3,0);
        cout << "end of try block" << endl;
    }
    catch(CException e) {
        cout << e.msg << endl;
    }
    cout << "f=" << f << endl;
    cout << "finished" << endl;
    return 0;
}
```

输出结果:

salary < 0
tax counted
devided by zero
f=1.2
finished

C++标准异常类

- C++标准库中有一些类代表异常，这些类都是从exception类派生而来



bad_cast

在用 `dynamic_cast` 进行从多态基类对象（或引用），到派生类的引用的强制类型转换时，如果转换是不安全的，则会抛出此异常。

```
#include <iostream>
#include <stdexcept>
#include <typeinfo>
using namespace std;
class Base
{
    virtual void func(){}
};
class Derived : public Base
{
public:
    void Print() { }
};
```

```
void PrintObj( Base & b)
{
    try {
        Derived & rd =
dynamic_cast<Derived&>(b);
        //此转换若不安全, 会抛出bad_cast异常
        rd.Print();
    }
    catch (bad_cast& e) {
        cerr << "e.what()" << endl;
    }
}

int main ()
{
    Base b;
    PrintObj(b);
    return 0;
}
```

输出结果:

Bad dynamic_cast!

bad_alloc

在用new运算符进行动态内存分配时，如果没有足够的内存，则会引发此异常。

```
#include <iostream>
#include <stdexcept>
using namespace std;
int main ()
{
    try {
        char * p = new char[0x7fffffff];
        //无法分配这么多空间，会抛出异常
    }
    catch (bad_alloc & e) {
        cerr << e.what() << endl;
    }
    return 0;
}
```

输出结果:

bad allocation

out_of_range

用vector或string的at成员函数根据下标访问元素时，如果下标越界，就会抛出此异常。例如：

```
#include <iostream>
#include <stdexcept>
#include <vector>
#include <string>
using namespace std;
int main ()
{
    vector<int> v(10);
    try {
        v.at(100)=100;    //抛出out_of_range异常
    }
    catch (out_of_range& e) {
        cerr << e.what() << endl;
    }
}
```

```
string s = "hello";  
try {  
    char c = s.at(100); //抛出out_of_range异常  
}  
catch (out_of_range& e) {  
    cerr << e.what() << endl;  
}  
return 0;  
}
```

输出结果:

```
invalid vector<T> subscript  
invalid string position
```